

Assignment 1: KNN with threads, OpenMP, and MPI

9/20/2026

100 Points Possible

Attempt 1



In Progress

NEXT UP: Submit assignment

Add comment

Unlimited Attempts Allowed

9/20/2026

Details

Machine learning involves the use of datasets with millions of instances. The computational complexity of machine learning methods limits their scalability to large-scale datasets. The simplest classifier is the k-nearest-neighbor classifier ([KNN \(https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm\)](https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm)) but its computational complexity is $O(\text{num_test} \times \text{num_train} \times \text{num_features})$, where `num_test` and `num_train` are the number of instances of the test and train datasets, respectively, and `num_features` is the dimensionality of the feature space.

The KNN algorithm computes the distances to all of the train instances for each test instance and predicts the data class by majority voting from the k-nearest neighbors. The accuracy of the classifier is measured as the relation between the number of successful predictions and the number of instances. You may run the serial code provided below and analyze how the runtime increases with the size of the dataset.

The assignment consists of implementing parallel and distributed versions of the KNN algorithm. First, a multi-threaded version using POSIX threads. Second, a multi-threaded version using OpenMP. Third, a distributed version using MPI. The serial single-thread version is provided to facilitate the reading of the dataset files and the accuracy calculation. The number of k should be defined by an input parameter in the function and provided by the user (typically $k = 3$). The goal is not to improve the accuracy (accuracy **must** remain the same on the different versions) but to improve the scalability and reduce the runtime using parallel and distributed computing.

Activities:

1. Download the [assignment \(https://viriniacommonwealth.instructure.com/courses/137487/files/19504001?wrap=1\)](https://viriniacommonwealth.instructure.com/courses/137487/files/19504001?wrap=1), including a fully functional serial code, templates for the parallel and distributed versions, and scripts to compile the code.
2. Read the serial.cpp code, understand the code, and run it on **maple.cs.vcu.edu** using the small, medium, and large datasets.

3. Implement a parallel version of the algorithm: 1) using POSIX threads, 2) using OpenMP, and 3) using MPI. Code templates are provided.
4. Run the code on **maple.cs.vcu.edu** using 1, 2, 4, 8, 16, 32, 64, and 128 threads/processes on the small, medium, and large datasets, respectively.
5. Write the Slurm scripts to run the code on **athena.hprc.vcu.edu** and then run the parallel code using 1, 2, 4, 8, 16, 32, 64, and 128 threads/processes on the small, medium, and large datasets, respectively. Specifically, for MPI compare as well when the execution is forced on 1 node only vs any number of nodes.
6. Run each experiment 3 times and calculate the average runtimes to get more accurate estimates.
7. Write a report 1) describing the approach you followed to implement each solution; 2) include an analysis of the results (tables and figures with runtimes, speedups, estimation of the proportion of parallel code, etc).

Important: use **maple.cs.vcu.edu** to develop, debug, and test your code using Visual Studio Code. Only run the complete experiments on **athena.hprc.vcu.edu** when your code is error-free. You can run the code on **maple.cs.vcu.edu** anytime. However, be mindful of other students using the machine at the same time. Since Athena is a shared cluster there is no guarantee you will be able to run experiments if you wait too close to the deadline. Work early!

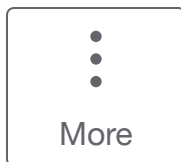
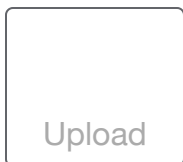
Deliverables:

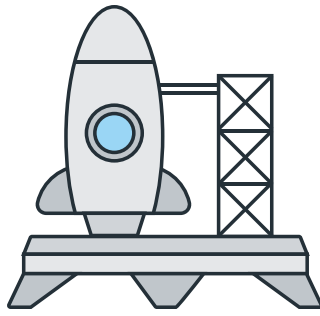
1. Source code. Complete the provided template files with the code for the threaded, OpenMP, and MPI versions.
2. Slurm scripts to run (correctly) the code on athena.hprc.vcu.edu.
3. Report with the methodology followed and results (including tables and figures).

Compress the source code (all files including code, scripts, datasets, and Makefile) and the report into a .zip file. Upload the .zip file to Canvas.

Warning: cheating/plagiarism of the assignment will be prosecuted. You've been warned.

Choose a submission type





Choose a file to upload
File permitted: ZIP

or

 Canvas Files

Submit assignment